

ft_blockchain: Estructuras de Datos y Colecciones

1. Introducción al Concepto

En C, para almacenar datos variables sufrías creando listas enlazadas dinámicas y manejando punteros dobles. Java proporciona el 'Java Collections Framework', una infraestructura masiva de estructuras de datos optimizadas. En este proyecto, aprenderás a elegir y utilizar estas colecciones construyendo un libro de contabilidad inmutable y distribuido: una mini Blockchain. Comprenderás el coste computacional de cada estructura y cómo los genéricos permiten la reutilización de código.

2. Especificaciones Obligatorias

- Modelar una clase 'Transaccion' (emisor, receptor, cantidad).
- Modelar una clase 'Bloque' que contenga un índice, una marca de tiempo, el hash del bloque anterior, su propio hash y un conjunto de transacciones.
- El hash de cada bloque debe calcularse utilizando el algoritmo criptográfico SHA-256 (disponible en 'java.security.MessageDigest').
- Crear una clase 'Blockchain' que almacene la cadena de bloques y valide de principio a fin que ningún bloque haya sido alterado (integridad de la cadena).
- Implementar un sistema de búsqueda eficiente para encontrar todas las transacciones asociadas a una cuenta específica.

3. Restricciones y Prohibiciones

- Queda estrictamente prohibido usar arrays nativos ('[]') para almacenar los bloques o las transacciones. Debes usar colecciones del framework.
- No puedes usar librerías externas de blockchain o hashing. La concatenación de datos y la llamada a MessageDigest deben ser manuales.



Evaluación de la Moulinette

La evaluación inyectará bloques corruptos en medio de la cadena para verificar si tu método de validación los detecta instantáneamente. Además, se medirá la eficiencia de búsqueda: usar una estructura inadecuada (como buscar linealmente en vez de usar mapas) reducirá tu nota.

4. Resumen del Aprendizaje

Al dominar Lists, Maps y Sets, desbloqueas el verdadero potencial de Java para manejar grandes volúmenes de información de manera eficiente y elegante.